

Demostración · La centralita y el tablón de anuncios

Service Discovery y Config Server · Guía del instructor · Curso de Spring Boot · SII
2026

20 minutos proyectando · Eureka y Config Server · el mismo sistema del Lab de
Microservicios

Resumen

Que las dos preguntas que cada servicio tenía escritas dentro —«¿dónde está el otro?» y «¿cuál es mi configuración?» — se le pueden dar a otro para que las conteste; que eso permite **mover un servicio de puerto sin tocar a nadie más y cambiar una propiedad sin recompilar ni reiniciar**; y que el registro que todo el mundo teme como punto único de fallo **aguanta cinco minutos muerto sin que el usuario vea un solo error** — medido, no dicho.

Índice

1. Antes de empezar	2
1.1. Esto no es un laboratorio, y además es opcional	2
1.2. Qué hay que dejar preparado	2
2. El caso	3
2.1. La centralita, que es la metáfora de esta demostración	3
2.2. Y el tablón de anuncios	3
3. Cómo se registran los servicios	4
4. El guion	4
4.1. Bloque 1 · «¿Quién existe?» — 4 minutos	4
4.2. Bloque 2 · Llamar por nombre — 5 minutos	5
4.3. Bloque 3 · La configuración, fuera del programa — 4 minutos	8
4.4. Bloque 4 · Apagar el registro — 5 minutos	9
4.5. Bloque 5 · El cierre — 2 minutos	11
5. Lo que este material NO trae	11
6. Los números, para responder sin dudar	12

1 Antes de empezar

1.1 Esto no es un laboratorio, y además es opcional

El alumno no corre nada de esto. Se proyecta, se mira, y se vuelve al laboratorio.

Y va más lejos que las otras demostraciones: **ésta es prescindible**. Si el tiempo aprieta, si algo no arranca, o si la sala viene espesa, el Lab de Microservicios y la demostración con Docker cubren la sesión enteros. **Nada de aquí es requisito de nada**. Va al final por eso.

Por la misma razón, esta guía **no tiene pasos para pegar**: no hay nada que teclear. Lo que tiene es **qué señalar y en qué orden**.

1.2 Qué hay que dejar preparado

Requisito	Cómo lo compruebas	Qué tiene que salir
Los seis servicios compilados	<code>./construir.sh --sin-red</code>	Termina en verde sin bajar nada
El sistema arriba	<code>./levantar.sh</code>	«Sistema LISTO en 25s»
El panel del registro, proyectado	<code>http://localhost:8761</code>	Cinco filas
<code>config-repo/gateway.yml</code> abierto en el editor		Se va a editar en directo

Si te atascas

Compila ANTES de la clase, con red.

Es lo único de esta demostración que necesita internet, y es la única vez. El resto del curso

compila offline contra `repo-maven/`; **Eureka y el Config Server no están ahí a propósito** —serían megas de dependencias en el clon de dieciocho alumnos para un material que ningún alumno ejecuta—, así que `construir.sh` sale a la red la primera vez y deja todo en la caché de Maven de tu máquina.

`./construir.sh --sin-red` es la comprobación del día antes: si pasa, la clase no va a necesitar wifi.

Levanta el sistema antes de que entre nadie, y déjalo arriba.

Son **veinticinco segundos** de pantalla parada. Se pueden enseñar una vez, a propósito, para que se vea lo que cuestan seis procesos. No se pueden regalar dos veces.

2 El caso

2.1 La centralita, que es la metáfora de esta demostración

La guía del Lab de Microservicios dejó a la DGT partida en cuatro oficinas: Contribuyentes con su padrón, Trámites con sus expedientes, Auditoría con su registro, y una recepción por la que entra el ciudadano.

Y cada oficina tiene, **pegada con celo en la pared**, una nota con los teléfonos de las otras tres.

Funciona. Funciona perfectamente, de hecho, **mientras nadie se muda**. El día que Contribuyentes cambia de planta, alguien tiene que ir oficina por oficina, despegar la nota, escribir el teléfono nuevo y volver a pegarla. Si se olvida una, esa oficina llamará durante meses a un número que ya no contesta — y no lo va a descubrir hasta que necesite llamar.

Un registro de servicios es contratar una centralita. Ya no hay notas en las paredes: cada oficina, al abrir por la mañana, llama a la centralita y dice «soy Contribuyentes y estoy en el teléfono tal». Cuando Trámites necesita a Contribuyentes, pregunta a la centralita. Si Contribuyentes se muda, **se lo dice a la centralita y nadie más se entera de nada** — que es exactamente el objetivo.

2.2 Y el tablón de anuncios

La segunda pieza es la otra mitad del mismo problema, y la metáfora es igual de vieja.

Cada oficina tenía también, **en un cajón**, su propia carpeta de instrucciones: a qué hora abrir, cuántos minutos esperar al teléfono, en qué archivador guardar. Cuatro carpetas, cuatro cajones. Para cambiar el horario de todas hay que abrir cuatro cajones — y para que el cambio surta efecto, cerrar la oficina y volver a abrirla.

Un Config Server es poner esas instrucciones en un tablón de anuncios común. Las carpetas desaparecen de los cajones. Y lo mejor no es tenerlas juntas: es que **se puede cambiar una instrucción con la oficina abierta y el público dentro**.

La abuelita, si te pregunta qué has enseñado hoy:

«Antes cada oficina tenía los teléfonos de las otras apuntados en un papel en la pared, y su horario en un cajón. Hoy hemos puesto una centralita y un tablón de anuncios. Ahora una oficina puede mudarse y las demás la siguen encontrando, y se puede cambiar el horario sin cerrar.

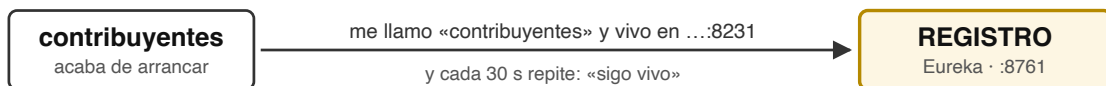
Y lo mejor: hemos apagado la centralita cinco minutos a ver qué pasaba, y no

pasó nada — porque cada oficina se había apuntado los teléfonos en su libreta.»

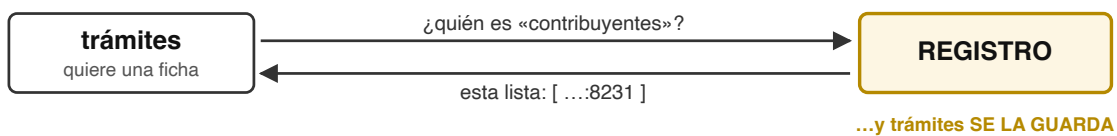
3 Cómo se registran los servicios

Este diagrama es el que hay que tener claro antes de proyectar nada, porque explica los cuatro bloques de la demostración y sobre todo el último.

1 · AL ARRANCAR — «existo, y estoy aquí»



2 · AL LLAMAR — «¿dónde está?»



3 · LA LLAMADA — el registro ya no pinta nada



Figura 1: El registro, en tres momentos

Los tres momentos, y el tercero es el que importa:

1. **Al arrancar**, el servicio se presenta: «me llamo así y vivo aquí». Y repite cada 30 segundos para decir que sigue vivo.
2. **Al llamar** a otro, pregunta por el nombre y recibe una **lista** de direcciones — que se **guarda en su propia memoria**.
3. **La llamada va directa**. El registro **no está en medio**. Sólo dijo la dirección, antes.

Ese tercer momento es la razón de que apagar el registro no corte el sistema, y es lo que el bloque 4 demuestra en vivo. Vale la pena dejarlo dicho ya, aunque se vaya a repetir.

4 El guion

4.1 Bloque 1 · «¿Quién existe?» — 4 minutos

Se proyecta `http://localhost:8761` y se lee la tabla en voz alta: cinco filas, una por servicio.

Qué señalar, en este orden:

1. **Nadie escribió esa tabla**. No hay ningún archivo con las cinco direcciones. Cada servicio se presentó solo al arrancar.
2. **El Config Server sale en la lista**, como uno más. La infraestructura no está exenta de las reglas que le impone al resto.
3. **Son cinco, y los procesos son seis**. El que falta es el registro: no se inscribe en sí mismo. Con varios nodos sí lo haría — así se replican entre ellos — y ahí está la respuesta a la pregunta que alguien hará en el bloque 4.

Y ahora, **en directo**, con el panel proyectado y otra terminal al lado:

```
./apagar.sh contribuyentes      # recargar el panel: quedan CUATRO
./levantar.sh contribuyentes    # recargar otra vez
```

Aparece a los 6 segundos del lanzamiento. Se lanza, se cuenta hasta seis, se recarga el navegador y está.

Vas bien si...

La pregunta que hay que provocar aquí, porque la respuesta es el resto de la sesión:

«Vale, ¿y si se cae esa cosa?»

Si nadie la hace, se hace uno mismo en voz alta y se deja en el aire: «lo vemos en cinco minutos».

Y una que sorprende

Al apagar contribuyentes con `./apagar.sh`, **desaparece del panel al instante**. No a los 90 segundos, que es lo que todo el mundo espera de Eureka.

Es que hay **dos maneras de morir**:

Apagado ordenado (SIGTERM, Ctrl+C, un despliegue normal)

Spring corre su gancho de cierre y el servicio **se da de baja él mismo**. El registro queda al día en milisegundos

Muerte dura (kill -9, sin memoria, la máquina se apaga)

Nadie avisa. El registro sólo puede notarlo por **ausencia de latidos**

Y esto se midió, con un `kill -9`:

```
t+ 0s    el registro TODAVÍA lo da por vivo (HTTP 200)
t+30s    el registro TODAVÍA lo da por vivo (HTTP 200)
t+61s    el registro TODAVÍA lo da por vivo (HTTP 200)
t+86s    el registro lo tacha
```

Ochenta y seis segundos dando por buena la dirección de un proceso que ya no existe.

Éste es el número que hay que dejar dicho, porque explica hacia atrás medio curso: el circuit breaker del Lab 10 y del Lab de Microservicios no está «por si acaso». Está porque **la lista es falible por diseño**, y todo el que llame a otro servicio tiene que estar preparado para que la dirección que le acaban de dar no conteste.

4.2 Bloque 2 · Llamar por nombre — 5 minutos

Es el bloque que hace visible el patrón.

El antes y el después, en el YAML

En el laboratorio, la dirección de contribuyentes:

```
microservicios:
  contribuyentes:
    url: http://localhost:8211
```

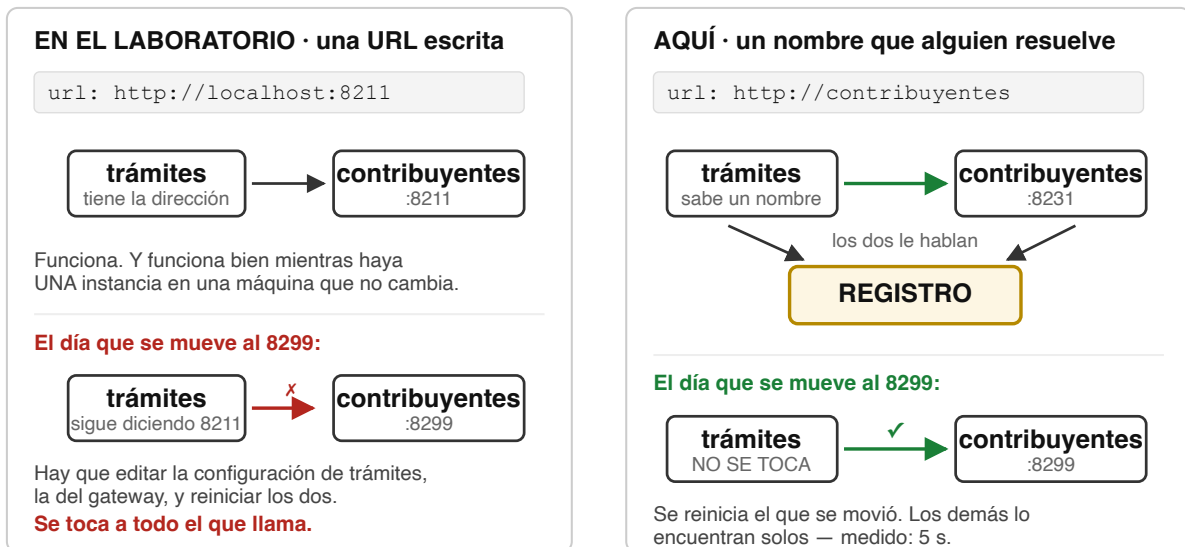


Figura 2: Lo que cambia entre una URL fija y un nombre

```

auditoria:
  url: http://localhost:8213
  
```

Y aquí:

```

microservicios:
  contribuyentes:
    url: http://contribuyentes
  auditoria:
    url: http://auditoria
  
```

Qué señalar: desapareció el puerto, y con él la máquina. Queda **un nombre**.

Y en el código, una línea

```

public ClienteContribuyentes(@Value("${microservicios.contribuyentes.url}") String
↪ url,
                               DeferringLoadBalancerInterceptor balanceador) {
    JdkClientHttpRequestFactory fabrica =
        new JdkClientHttpRequestFactory(HttpClient.newBuilder().connectTimeout(
            ↪ TIMEOUT).build());
    fabrica.setReadTimeout(TIMEOUT);
    this.http = RestClient.builder()
        .baseUrl(url)
        .requestFactory(fabrica)
        .requestInterceptor(balanceador)
        .build();

    this.circuito = CircuitBreaker.of("contribuyentes", CircuitBreakerConfig.custom()
        .slidingWindowType(CircuitBreakerConfig.SlidingWindowType.COUNT_BASED)
        .slidingWindowSize(4)
  
```

```

        .minimumNumberOfCalls(3)
        .failureRateThreshold(50)
        .waitDurationInOpenState(Duration.ofSeconds(10))
        .permittedNumberOfCallsInHalfOpenState(1)
        .ignoreExceptions(HttpClientErrorException.class)
        .build();

circuito.getEventPublisher().onStateTransition(e ->
    log.warn("[CIRCUITO] {} -> {}",
        e.getStateTransition().getFromState(),
        e.getStateTransition().getToState()));
}

```

Lo único nuevo es `.requestInterceptor(balancedador)`. Ese interceptor hace tres cosas en cada llamada, y la segunda es la que una URL fija no puede hacer:

1. pide al registro —**a la copia local**— las instancias que se llaman contribuyentes;
2. **elige una**; si hubiera varias, iría rotando — de ahí lo de «balanceador»;
3. reescribe la URL con el host y el puerto de la elegida.

Que ese nombre no lo resuelve el sistema operativo

```
ping -c1 contribuyentes
```

ping: cannot resolve contribuyentes: Unknown host

Aquí está la diferencia con la demostración de Docker, y merece pararse.

Allí contribuyentes también se resolvía por nombre — pero lo hacía **el DNS de la red del compose**, gratis, sin que nadie montara nada. Por eso allí Eureka sobraba y se dijo así.

Aquí **no hay plataforma debajo**: son seis JVM en un portátil. El nombre contribuyentes **sólo existe dentro del registro**.

Eureka no compite con Docker. Compite con no tener plataforma.

Quién lo resuelve, entonces

```
curl -s localhost:8232/a-quien-veo
```

```

{
  "fuente": "la copia local del registro que guarda este proceso",
  "servicios": {
    "contribuyentes": ["192.168.100.218:8231"],
    "tramites":       ["192.168.100.218:8232"],
    "auditoria":      ["192.168.100.218:8233"],
    "gateway":        ["192.168.100.218:8230"],
    "config":         ["192.168.100.218:8888"]
  }
}

```

Léase la palabra «copia» en voz alta. Trámites no pregunta a Eureka en cada llamada: se baja el registro entero cada pocos segundos y lo guarda. **Esa frase es la semilla del bloque 4.**

El remate: mover un servicio de sitio

Se edita `config-repo/contribuyentes.yml`, `port: 8231` → `port: 8299`, y se reinicia **sólo** contribuyentes. **Nadie toca trámites. Nadie toca el gateway. Nadie recompila.**

Y vuelve a funcionar. Pero **tarda 23 segundos**, y contarlo bien es la mitad del valor del bloque:

~5 s	trámites se baja un registro donde contribuyentes ya está en el 8299
~18 s	el circuit breaker , que se abrió con los primeros fallos y sólo reintenta cada 10 s

En el log de trámites, proyectado:

```
18:03:58.483 WARN [CIRCUITO] HALF_OPEN -> OPEN      <- reintentó demasiado pronto
18:04:09.389 WARN [CIRCUITO] OPEN -> HALF_OPEN
18:04:09.661 WARN [CIRCUITO] HALF_OPEN -> CLOSED    <- y se curó solo
```

Vas bien si...

El registro dijo la verdad en cinco segundos. El que tardó fue el circuit breaker.

Son dos mecanismos independientes, y el sistema va a la velocidad del más lento. Es el mismo patrón que la demostración con Docker enseña en su bloque 5b: el orquestador levanta el proceso en tres décimas, y **decidir cuándo volver a confiar en él sigue siendo del programa.**

4.3 Bloque 3 · La configuración, fuera del programa — 4 minutos

Primero, lo que ya no está dentro

Se abre el `application.yml` del gateway. **Entero, son cuatro líneas:**

```
spring:
  application:
    name: gateway
  config:
    import: "configserver:http://localhost:8888"
```

Qué señalar: en el laboratorio este archivo tenía el puerto, las tres direcciones, el secreto del JWT y el formato del log. Aquí quedan dos cosas irreducibles: **cómo se llama** —que es la pregunta que le hace al Config Server— y **dónde preguntar**. En algún punto la cadena toca suelo, y toca aquí.

El cambio en caliente

El instrumento es `GET /rutas`, que enseña la tabla del gateway tal como está **ahora**.

```
curl -s localhost:8230/rutas
```

Se edita `config-repo/gateway.yml` **proyectado**, delante de la sala:

```
tramites:
  url: http://tramites-que-no-existe
```


Y **antes de refrescar**, las dos comprobaciones que separan las dos mitades del asunto:

```
curl -s localhost:8888/gateway/default | grep tramites # el Config Server YA lo
↪ sirve
curl -s localhost:8230/rutas # el gateway NO se ha enterado
```

El Config Server no avisa a nadie. No hay notificación, ni sondeo, ni magia. Guardar el archivo no cambia nada en ningún servicio: **alguien tiene que pedir el cambio.**

```
curl -s -X POST localhost:8230/actuador/refresh
```

```
["microservicios.tramites.url"]
```

Ahí está la demostración entera, en esa respuesta: el endpoint dice, con nombre y apellido, qué propiedad ha cambiado. Y el efecto se ve en pantalla al instante — la petición que daba 200 pasa a dar 503. Se deshace, se refresca otra vez, y vuelve el 200 en **0,1 segundos**.

Sin recompilar, sin reiniciar, sin desplegar.

Lo que NO se recarga, que es la mitad honesta

Sí	Lo que lee un bean anotado con
No	@RefreshScope — aquí, TablaDeRutas
No	server.port. Lo usa el servidor web al
	arrancar y no vuelve a mirarlo. Exige reiniciar
	El puerto de la base embebida: está en el
	main(), antes de que exista el Environment

Si te atascas

@RefreshScope no es gratis ni automático. Es una anotación que alguien tiene que poner, en el bean concreto que lee la propiedad.

Un Config Server sin @RefreshScope sirve configuración centralizada, sí — pero para que llegue hay que reiniciar, y entonces media gracia se ha ido. **Si alguien pregunta «¿y esto se recarga solo?», la respuesta honesta es «lo que hayas marcado, y sólo cuando alguien lo pida».**

4.4 Bloque 4 · Apagar el registro — 5 minutos

Es el bloque que justifica la demostración entera. Y responde a la pregunta que quedó en el aire en el bloque 1.

```
./apagar.sh registro
```

Y se deja una sonda corriendo, sin tocar nada más. **Medido, cinco minutos seguidos:**

```
t+ 0s HTTP 200 nombre=OK tramites-ve=4 registro=(no responde)
t+ 60s HTTP 200 nombre=OK tramites-ve=4 registro=(no responde)
t+180s HTTP 200 nombre=OK tramites-ve=4 registro=(no responde)
t+303s HTTP 200 nombre=OK tramites-ve=4 registro=(no responde)
```

Cinco minutos. Ni un solo error. El usuario no se entera de nada. Y se paró la medición, no el sistema.

Por qué, y es la frase del bloque: porque **el registro no está en el camino de ninguna petición** — el tercer momento del diagrama. Al registro sólo se le pregunta para actualizar la copia, y si no contesta, cada uno se queda con la última que tenía y sigue trabajando.

Lo único que pasa es ruido en el log, cada cinco segundos:

```
INFO TRAMITES - was unable to refresh its cache! ... retried in 5 seconds
```

Y ahora, qué **SÍ** se rompe — que es la otra mitad

Un registro caído **no se nota mientras nada se mueva**. Con Eureka todavía muerto, se mueve contribuyentes al 8299 y se reinicia:

```
contribuyentes vivo y sano en :8299
(pero no ha podido inscribirse en ningún sitio)
```

```
t+15s estadoDelNombre=NO_DISPONIBLE trámites sigue viendo ...:8231
t+60s estadoDelNombre=NO_DISPONIBLE trámites sigue viendo ...:8231
```

Contribuyentes está perfectamente vivo y trámites no lo va a encontrar nunca, porque el único que podía contárselo está muerto. Y no se cura solo.

Vas bien si...

La conclusión, y es más fina que «Eureka no es un punto único de fallo»:

Un registro caído no rompe lo que ya funcionaba — **congela** el sistema en la foto que cada uno tenía. Lo que se pierde no es el tráfico: es la **capacidad de cambiar**. Nada se puede desplegar, ni mover, ni escalar, ni sustituir, porque nadie se va a enterar.

El registro no está en el camino de las peticiones. Está en el camino de los despliegues.

Y por eso en producción se corren **varios nodos** de Eureka replicándose entre ellos — que es, por cierto, la razón de que un Eureka Server sea cliente de sí mismo, esa casilla que en el bloque 1 estaba apagada.

Volver a encenderlo, que sorprende más que apagarlo

Si te atascas

Decisión de guion: esto probablemente NO se hace en vivo. Son 45 segundos de pantalla en rojo. Se cuenta con la tabla delante, y si sobra tiempo se hace.

```
t+ 4s NO_DISPONIBLE trámites ve ...:8231      <- la dirección VIEJA
t+14s NO_DISPONIBLE trámites ve (nada)         <- ise quedó sin nada!
t+24s NO_DISPONIBLE trámites ve ...:8299      <- ya lo ve bien
t+45s OK                                           <- y por fin sirve
```

0 □ 4 s

arranca el registro, **vacío**: no recuerda nada de antes de morir

4 □ 14 s

los clientes se bajan ese registro vacío y **borran su copia buena**

14 □ 24 s

los servicios notan que no los conoce y **se vuelven a inscribir** solos el **circuit breaker**, otra vez

24 □ 45 s

La línea de t+14s es la que hay que señalar con el dedo.

Un registro que vuelve **vacío** deja el sistema momentáneamente **peor** que teniéndolo apagado: apagado, cada uno conservaba su última copia buena; encendido y vacío, todos se creen la lista nueva —que no tiene a nadie— y tiran la que servía.

Es contraintuitivo, es real, y es exactamente la razón de existir del **modo de autopreservación** de Eureka, que en esta demostración está apagado a propósito para que el panel sea legible.

4.5 Bloque 5 · El cierre — 2 minutos

```
./apagar.sh
```

Lo que hay que dejar dicho, y es lo contrario de una venta:

Las dos piezas resolvieron dos problemas **reales**: las direcciones dejaron de estar escritas a mano en cuatro sitios, y la configuración dejó de estar dentro del artefacto. Con **una** instancia de cada servicio en **una** máquina, ninguno de los dos dolía — y por eso el laboratorio no las lleva. Con instancias que se crean y se destruyen solas, los dos son el trabajo del día.

Y lo que se pagó, sin restar nada:

- **Dos procesos más** que arrancar, vigilar, actualizar y explicar.
- **Un requisito de arranque nuevo**: sin el Config Server, los cuatro servicios **no arrancan**.
- **Veinticinco segundos** hasta la primera petición.
- **Configuración con relojes**: 5 s de refresco, 30 s de latido, 90 s de expiración. Números que antes no existían y que ahora hay que conocer para entender por qué algo tarda.

Vas bien si...

La pregunta con la que se cierra, y que el alumno se va a encontrar de verdad:

No es «¿registro sí o no?». Es «¿ya tengo una plataforma que me lo dé?».

Si vas a desplegar sobre Kubernetes o sobre cualquier orquestador, el descubrimiento y buena parte de la configuración **ya te vienen dados** — lo viste en la demostración con Docker, con getent hosts. Montar un Eureka encima es duplicar.

Si despliegas seis JVM en dos máquinas que administras tú —que es donde está media banca y media administración pública—, esto es **exactamente** lo que necesitas.

5 Lo que este material NO trae

Tan importante como lo anterior, y va dicho en clase:

- **Balanceo entre N instancias**. Es la mejor demostración posible de un registro —dos contribuyentes, el mismo nombre, las peticiones repartiéndose— y **no está**: serían siete procesos y el laboratorio no balancea. Es lo primero que habría que añadir.

- **Varios nodos de Eureka replicándose.** Se explica de palabra en el bloque 4; montarlo son dos procesos más.
- **spring-cloud-bus**, que propaga un refresh a todos los servicios de golpe en vez de uno a uno. Necesita un broker de mensajes, y eso no cabe en la maleta.
- **Backend Git para el Config Server.** Aquí son archivos, para poder editarlos proyectados. En producción es un repositorio Git — y no por moda: se quiere el **historial** (quién bajó el timeout y cuándo) y la **reversión** (`git revert` sobre el cambio que tumbó el sistema anoche).
- **Seguridad en el Config Server.** Aquí sirve sus archivos a quien pregunte. Un Config Server real guarda credenciales y va cifrado y autenticado.

6 Los números, para responder sin dudar

Todos medidos en el Mac donde se preparó, tres corridas.

Arranque de los seis, hasta servir	24 – 27 s
Memoria total (6 JVM + 3 PostgreSQL)	≈ 2,4 GB
Aparecer en el panel	6,1 s
Desaparecer: apagado ordenado / <code>kill -9</code>	inmediato / 86 s
Encontrar un servicio que cambió de puerto	23 s (5 s el registro, 18 s el circuit breaker)
Recarga de configuración en caliente	inmediata
Aguante con el registro muerto	5 min sin un error
Recuperación al volver el registro	45 s
